



MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Daniel Moisei, FAF-241

Report

Laboratory Work No. 1

Cryptography and Security

Topic: Caesar Cipher

Romanian Alphabet and Keyword-Based Permutation

Checked by:

Maia Zaica, *university assistant*

CONTENTS

Introduction	3
1 Short Theory	4
1.1 The Caesar Cipher	4
1.2 Caesar Cipher with a Permuted Alphabet	4
2 Implementation	5
2.1 Romanian Alphabet and Constraints	5
2.2 Encryption and Decryption	5
2.3 Building the Permuted Alphabet	6
2.4 Normalization and Input Validation	6
3 Results	8
3.1 Task 1.1: Standard Romanian Alphabet	8
3.2 Task 1.2: Keyword-Permuted Alphabet	8
3.3 Invalid Input	9
4 Conclusions	11
5 Source Code	11

INTRODUCTION

The purpose of this laboratory work is to understand and implement the Caesar cipher, evaluate the effect of its small key space, and extend it by permuting the alphabet with a keyword. Both versions operate on the 31-letter Romanian alphabet and use the explicit letter encoding required by the laboratory handout rather than Unicode code-point arithmetic.

The application was implemented in C# as an interactive console program. It supports encryption and decryption, normalizes Romanian letters, removes spaces from messages, validates all user input, and displays the active alphabet so that the result can be checked manually. The implementation also accepts the cedilla variants Ș and Ț and converts them to the standard Romanian comma-below forms Ș̣ and Ț̣.

1 SHORT THEORY

1.1 The Caesar Cipher

The Caesar cipher is a monoalphabetic substitution cipher. Each plaintext letter is replaced by the letter located a fixed number of positions later in the alphabet. If a letter is close to the end, the position wraps around to the beginning. Let n be the alphabet length, k the numeric key, x the plaintext letter code, and y the ciphertext letter code. Encryption and decryption are defined by

$$y = e_k(x) = (x + k) \bmod n, \quad x = d_k(y) = (y - k) \bmod n. \quad (1)$$

For the Romanian alphabet, $n = 31$ and $k \in \{1, \dots, 30\}$. Therefore, the ordinary cipher has only 30 possible keys. An attacker can try all of them quickly and identify the readable plaintext, so the cipher does not provide practical confidentiality.

The encoding used in the program is

AĂÂBCDEFGHIÎJKLMNOPSŞŢȚUVWXYZ,

where the position of a letter in the string is its numeric code, starting from zero.

1.2 Caesar Cipher with a Permuted Alphabet

The second version uses two keys. Key k_1 is the numeric shift from 1 to 30, while key k_2 is a keyword containing at least seven Romanian letters. The keyword is normalized to uppercase, repeated letters are retained only at their first occurrence, and the unused letters of the standard alphabet are appended. This produces the permuted alphabet.

The same formulas are then applied, but x and y refer to positions in the permuted alphabet. In the idealized case there are $31!$ possible alphabet orders and 30 nonzero shifts, giving at most

$$31! \times 30 \quad (2)$$

key pairs. Some different keyword/shift pairs may yield the same substitution. Although the enlarged key space makes exhaustive key search much harder, the method remains a monoalphabetic substitution: each plaintext letter always maps to the same ciphertext letter. Consequently, it is still vulnerable to frequency analysis and known-plaintext attacks.

2 IMPLEMENTATION

2.1 Romanian Alphabet and Constraints

The required alphabet and validation limits are declared as constants. The alphabet string itself is the code table: `IndexOf` returns the numeric code of a letter without using its Unicode value.

Listing 1: Alphabet encoding and key constraints

```
1 private const string Alphabet =  
2     "AĂÂBCDEFGHIÎJKLMNOPSȘȚTUVWXYZ";  
3 private const int MinimumShift = 1;  
4 private const int MaximumShift = 30;  
5 private const int MinimumKeywordLength = 7;
```

This representation guarantees that the implementation follows the exact order from the handout and makes the alphabet length available directly through `Alphabet.Length`.

2.2 Encryption and Decryption

Both operations are implemented by one transformation method. Encryption uses direction $+1$, while decryption uses -1 . Adding the alphabet length before the modulo operation prevents a negative array index during decryption.

Listing 2: Common transformation used for encryption and decryption

```
1 private static string Transform(  
2     string text, int shift, string activeAlphabet, bool encrypt)  
3 {  
4     var result = new StringBuilder(text.Length);  
5     int direction = encrypt ? 1 : -1;  
6  
7     foreach (char letter in text)  
8     {  
9         int index = activeAlphabet.IndexOf(letter);  
10        int newIndex = (index + direction * shift  
11                        + activeAlphabet.Length)  
12                        % activeAlphabet.Length;  
13        result.Append(activeAlphabet[newIndex]);  
14    }  
15  
16    return result.ToString();  
17 }
```

Because validation is performed before this method is called, every character is guaranteed to have a valid index in the active alphabet.

2.3 Building the Permuted Alphabet

The keyword is concatenated with the standard Romanian alphabet. LINQ's `Distinct` preserves the first occurrence of every character, which removes repeated keyword letters and then appends every missing alphabet letter exactly once.

Listing 3: Construction of the keyword-permuted alphabet

```
1 private static string BuildPermutedAlphabet(string keyword)
2 {
3     return new string(
4         (keyword + Alphabet).Distinct().ToArray());
5 }
```

For example, the keyword `CRIPTOGRAFIE` produces

`CRIPTOGA FEĂĂBDH ÎJKLMNQSSȚUVWXYZ.`

The program prints this value before transforming the message, as required by Task 1.2.

2.4 Normalization and Input Validation

Input is normalized to Unicode Form C and uppercase. Cedilla variants are converted to the Romanian comma-below forms so that input from different keyboards is accepted consistently.

Listing 4: Normalization of Romanian letters

```
1 private static string NormalizeRomanianLetters(string value)
2 {
3     return value
4         .Normalize(NormalizationForm.FormC)
5         .ToUpperInvariant()
6         .Replace('ș', 'Ș')
7         .Replace('ț', 'Ț');
8 }
```

The numeric key reader repeats until parsing succeeds and the value lies in the inclusive range 1–30.

Listing 5: Numeric-key validation

```
1 if (int.TryParse(input, out int shift)
2     && shift is >= MinimumShift and <= MaximumShift)
3 {
4     return shift;
5 }
6
7 Console.WriteLine(
8     "Invalid key. Enter an integer from 1 to 30 inclusive.");
```

Message validation allows only Romanian alphabet letters and spaces. It reports the rejected character, removes all spaces, and rejects an empty message. Keyword validation similarly reports unsupported characters and requires a minimum length of seven letters.

Listing 6: Validation of message characters

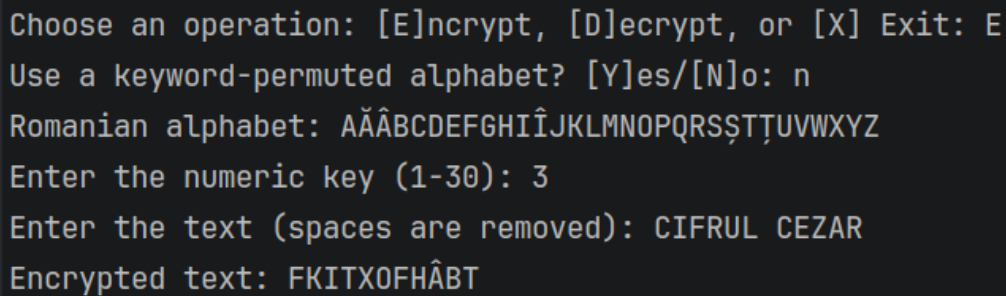
```
1 char invalidCharacter = text.FirstOrDefault(  
2     character => character != ' '  
3         && !Alphabet.Contains(character));  
4  
5 if (invalidCharacter != '\0')  
6 {  
7     Console.WriteLine(  
8         $"Invalid text character: '{invalidCharacter}'. " +  
9         $"Use only spaces and Romanian alphabet letters: " +  
10        Alphabet + ".");  
11     continue;  
12 }
```

3 RESULTS

The following tests cover successful encryption and decryption for both tasks, followed by the invalid-input cases requested in the handout. The screenshot slots compile as placeholders until the corresponding PNG files are placed in the `figures` directory.

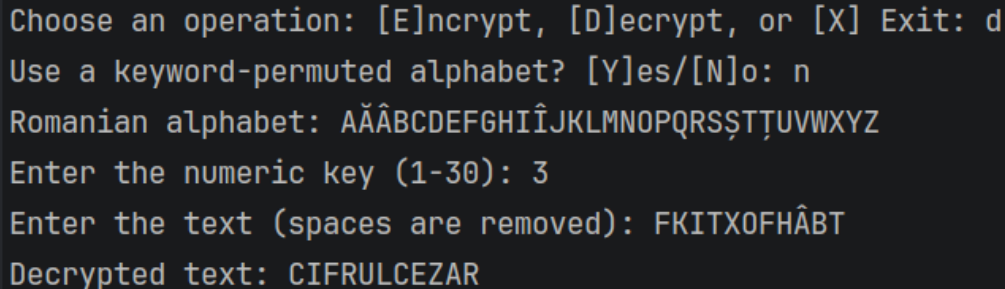
3.1 Task 1.1: Standard Romanian Alphabet

With plaintext `CIFRUL CEZAR` and $k = 3$, the program removes the space and produces `FKITXOFHÂBT`. Decrypting that ciphertext with the same key recovers `CIFRULCEZAR`.

A screenshot of a terminal window with a dark background and light-colored text. The text shows the program's prompts and user input for encryption: 'Choose an operation: [E]ncrypt, [D]ecrypt, or [X] Exit: E', 'Use a keyword-permuted alphabet? [Y]es/[N]o: n', 'Romanian alphabet: AĂÂBCDEFGHIÎJKLMNOPSŞŢTUVWXYZ', 'Enter the numeric key (1-30): 3', 'Enter the text (spaces are removed): CIFRUL CEZAR', and 'Encrypted text: FKITXOFHÂBT'.

```
Choose an operation: [E]ncrypt, [D]ecrypt, or [X] Exit: E
Use a keyword-permuted alphabet? [Y]es/[N]o: n
Romanian alphabet: AĂÂBCDEFGHIÎJKLMNOPSŞŢTUVWXYZ
Enter the numeric key (1-30): 3
Enter the text (spaces are removed): CIFRUL CEZAR
Encrypted text: FKITXOFHÂBT
```

Figure 3.1: Task 1.1 encryption with the standard Romanian alphabet

A screenshot of a terminal window with a dark background and light-colored text. The text shows the program's prompts and user input for decryption: 'Choose an operation: [E]ncrypt, [D]ecrypt, or [X] Exit: d', 'Use a keyword-permuted alphabet? [Y]es/[N]o: n', 'Romanian alphabet: AĂÂBCDEFGHIÎJKLMNOPSŞŢTUVWXYZ', 'Enter the numeric key (1-30): 3', 'Enter the text (spaces are removed): FKITXOFHÂBT', and 'Decrypted text: CIFRULCEZAR'.

```
Choose an operation: [E]ncrypt, [D]ecrypt, or [X] Exit: d
Use a keyword-permuted alphabet? [Y]es/[N]o: n
Romanian alphabet: AĂÂBCDEFGHIÎJKLMNOPSŞŢTUVWXYZ
Enter the numeric key (1-30): 3
Enter the text (spaces are removed): FKITXOFHÂBT
Decrypted text: CIFRULCEZAR
```

Figure 3.2: Task 1.1 decryption recovering the normalized plaintext

3.2 Task 1.2: Keyword-Permuted Alphabet

Using keyword `CRIPTOGRAFIE`, numeric key $k_1 = 5$, and plaintext `CIFRUL CEZAR`, the displayed permuted alphabet is `CRIPTOGAFEĂÂBDHÎJKLMNQSSŢTUVWXYZ`. Encryption produces `OADGZŞOHTBG`; decryption with the same two keys returns `CIFRULCEZAR`.

```

Choose an operation: [E]ncrypt, [D]ecrypt, or [X] Exit: e
Use a keyword-permuted alphabet? [Y]es/[N]o: y
Enter the permutation keyword (at least 7 Romanian letters): CRIPTOGRAFIE
Permuted alphabet: CRIPTOGAFEĂÂBDHÎJKLMNQSSŢUVWXYZ
Enter the numeric key (1-30): 5
Enter the text (spaces are removed): CIFRUL CEZAR
Encrypted text: OADGZŞOHTBG

```

Figure 3.3: Task 1.2 encryption and the displayed keyword-permuted alphabet

```

Choose an operation: [E]ncrypt, [D]ecrypt, or [X] Exit: d
Use a keyword-permuted alphabet? [Y]es/[N]o: y
Enter the permutation keyword (at least 7 Romanian letters): CRIPTOGRAFIE
Permuted alphabet: CRIPTOGAFEĂÂBDHÎJKLMNQSSŢUVWXYZ
Enter the numeric key (1-30): 5
Enter the text (spaces are removed): OADGZŞOHTBG
Decrypted text: CIFRULCEZAR

```

Figure 3.4: Task 1.2 decryption with the same numeric key and keyword

3.3 Invalid Input

An invalid numeric key causes the program to state that only integers from 1 to 30 are accepted and to ask again. An unsupported message character is displayed explicitly. For Task 1.2, a keyword shorter than seven letters is rejected, and a keyword containing a non-Romanian character is handled by the same validation loop.

```

Choose an operation: [E]ncrypt, [D]ecrypt, or [X] Exit: e
Use a keyword-permuted alphabet? [Y]es/[N]o: n
Romanian alphabet: AĂÂBCDEFGHIÎJKLMNQPQRSSŢŢUVWXYZ
Enter the numeric key (1-30): 0
Invalid key. Enter an integer from 1 to 30 inclusive.
Enter the numeric key (1-30): 92
Invalid key. Enter an integer from 1 to 30 inclusive.
Enter the numeric key (1-30): z
Invalid key. Enter an integer from 1 to 30 inclusive.
Enter the numeric key (1-30): █

```

Figure 3.5: Rejection of a numeric key outside the allowed range

```
Invalid key. Enter an integer from 1 to 30 inclusive.  
> Enter the numeric key (1-30): 3  
Enter the text (spaces are removed): yx ты  
Invalid text character: 'Y'. Use only spaces and Romanian alphabet letters: AĂÂBCDEFGHIÎJKLMNŌPQRSȘȚȚUVWXYZ.  
> Enter the text (spaces are removed):
```

Figure 3.6: Rejection of an invalid message character

```
Choose an operation: [E]ncrypt, [D]ecrypt, or [X] Exit: d  
> Use a keyword-permuted alphabet? [Y]es/[N]o: y  
Enter the permutation keyword (at least 7 Romanian letters): hello  
Invalid keyword. It must contain at least 7 letters.  
> Enter the permutation keyword (at least 7 Romanian letters):
```

Figure 3.7: Rejection of a keyword shorter than seven letters

4 CONCLUSIONS

This laboratory work demonstrated that modular arithmetic can implement encryption and decryption over a custom alphabet without relying on character-code arithmetic. Using one ordered string as the encoding table made the program correspond directly to the required 31-letter Romanian alphabet. Normalization and validation were essential because Romanian letters can be entered in multiple Unicode forms and invalid data must never reach the transformation step.

The ordinary Caesar cipher is insecure because it has only 30 possible nonzero shifts for the Romanian alphabet, so exhaustive search is immediate. A keyword-permuted alphabet greatly increases the apparent key space and prevents a simple 30-key search when the keyword is unknown. Nevertheless, it does not change the monoalphabetic nature of the cipher, and repeated letter-frequency patterns remain visible. The improved variant is therefore useful for learning about key spaces and substitutions, but it is not suitable for protecting real information.

5 SOURCE CODE

The complete C# implementation is available in the course repository:

<https://github.com/Moisei-D/CS-Labs/tree/main/labs/Lab01>

The main implementation is located in `Caesar.cs`, while `Program.cs` contains the console entry point.